

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT
TP. HỒ CHÍ MINH
Khoa Điện - Điện tử
Bộ môn Điện tử - Thông tin

Thực hành Học máy và Trí tuệ nhân tạo

Bài 4: CNNs for Image Classification

Giảng viên: TS. Huỳnh Thế Thiện & PGS. TS Trương Ngọc Sơn

Tháng 03, 2026

1 Mục tiêu

Sau khi hoàn thành bài thực hành này, sinh viên có thể:

- Hiểu nguyên lý hoạt động của Convolutional Neural Networks
- Xây dựng mô hình CNN bằng PyTorch
- Huấn luyện mô hình cho bài toán phân loại ảnh
- Hiểu vai trò của convolution, pooling và fully connected layers
- Cải thiện mô hình CNN

2 Giới thiệu bài toán

Trong bài thực hành này chúng ta giải quyết bài toán:

Phân loại ảnh

Dataset sử dụng là:

CIFAR-10

Dataset gồm:

- 60,000 ảnh màu
- kích thước ảnh: 32x32

- 10 classes

Các classes bao gồm:

- airplane
- automobile
- bird
- cat
- deer
- dog
- frog
- horse
- ship
- truck

3 Chuẩn bị môi trường

```
import torch
import torch.nn as nn
import torch.optim as optim

import torchvision
import torchvision.transforms as transforms

import matplotlib.pyplot as plt
import numpy as np
```

4 Tải dataset

```
transform = transforms.Compose(
[
transforms.ToTensor(),
transforms.Normalize((0.5,0.5,0.5),
                      (0.5,0.5,0.5))
]
)

trainset = torchvision.datasets.CIFAR10(
root='./data',
train=True,
```

```

download=True,
transform=transform
)

trainloader = torch.utils.data.DataLoader(
trainset,
batch_size=64,
shuffle=True
)

testset = torchvision.datasets.CIFAR10(
root='./data',
train=False,
download=True,
transform=transform
)

testloader = torch.utils.data.DataLoader(
testset,
batch_size=64,
shuffle=False
)

```

5 Hiển thị một số ảnh

```

classes = (
'airplane', 'automobile', 'bird', 'cat', 'deer',
'dog', 'frog', 'horse', 'ship', 'truck'
)

dataiter = iter(trainloader)
images, labels = next(dataiter)

plt.imshow(np.transpose(images[0], (1,2,0)))
plt.title(classes[labels[0]])
plt.show()

```

6 Xây dựng mô hình CNN

```

class CNN(nn.Module):

```

```

def __init__(self):

    super(CNN, self).__init__()

    self.conv1 = nn.Conv2d(
        in_channels=3,
        out_channels=32,
        kernel_size=3,
        padding=1
    )

    self.conv2 = nn.Conv2d(
        32,
        64,
        3,
        padding=1
    )

    self.pool = nn.MaxPool2d(
        kernel_size=2,
        stride=2
    )

    self.fc1 = nn.Linear(
        64 * 8 * 8,
        512
    )

    self.fc2 = nn.Linear(
        512,
        10
    )

    self.relu = nn.ReLU()

def forward(self, x):

    x = self.pool(self.relu(self.conv1(x)))

    x = self.pool(self.relu(self.conv2(x)))

```

```

x = x.view(-1, 64 * 8 * 8)

x = self.relu(self.fc1(x))

x = self.fc2(x)

return x

```

Khởi tạo mô hình:

```
model = CNN()
```

7 Loss function và Optimizer

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(
model.parameters(),
lr=0.001
)
```

8 Huấn luyện mô hình

```
epochs = 10
```

```

for epoch in range(epochs):

    running_loss = 0

    for images, labels in trainloader:

        outputs = model(images)

        loss = criterion(outputs, labels)

        optimizer.zero_grad()

        loss.backward()

```

```

optimizer.step()

running_loss += loss.item()

print("Epoch:", epoch,
      "Loss:", running_loss)

```

9 Đánh giá mô hình

```

correct = 0
total = 0

with torch.no_grad():

    for images, labels in testloader:

        outputs = model(images)

        _, predicted = torch.max(outputs,1)

        total += labels.size(0)

        correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total

print("Test accuracy:", accuracy)

```

10 Bài tập

10.1 Bài tập 1: Dataset Exploration

Thực hiện các yêu cầu:

- In kích thước train dataset
- In kích thước test dataset
- Hiển thị 10 ảnh đầu tiên

10.2 Bài tập 2: Hiểu kiến trúc CNN

Phân tích kiến trúc mạng:

- số convolution layers
- số filters của mỗi layer
- kích thước feature maps

10.3 Bài tập 3: Thay đổi số filter

Thử các cấu hình:

- $16 \rightarrow 32$ filters
- $32 \rightarrow 64$ filters
- $64 \rightarrow 128$ filters

So sánh accuracy.

10.4 Bài tập 4: Thêm convolution layer

Thêm 1 convolution layer vào mạng.

Ví dụ:

- conv1
- conv2
- conv3

So sánh kết quả.

10.5 Bài tập 5: Dropout

Thêm dropout vào mạng:

```
self.dropout = nn.Dropout(0.5)
```

So sánh accuracy trước và sau khi dùng dropout.

10.6 Bài tập 6: Batch Size

Thử các batch size:

- 32
- 64
- 128

So sánh:

- thời gian huấn luyện
- accuracy

10.7 Bài tập 7: Learning Rate

Thử các learning rate:

- 0.01
- 0.001
- 0.0001

Quan sát ảnh hưởng đến loss.

10.8 Bài tập 8: Data Augmentation

Thêm data augmentation:

```
transforms.RandomHorizontalFlip()  
transforms.RandomRotation(10)
```

So sánh kết quả.

10.9 Bài tập 9: Confusion Matrix

Tính confusion matrix cho mô hình.

Sử dụng:

```
from sklearn.metrics import confusion_matrix
```

Vẽ heatmap.

10.10 Bài tập 10: Cải thiện mô hình

Sinh viên cần:

- thử kiến trúc CNN khác
- thêm convolution layers
- thêm dropout
- dùng data augmentation

Mục tiêu:

Đạt accuracy > 75%